

Personalized Networking Assistant

Project Description:

The Personalized Networking Assistant is an AI-powered web application that helps users generate smart, tailored conversation starters for professional or social networking events. It uses DistilBERT for extracting themes from event descriptions and GPT-2 for generating context-aware conversation prompts. The platform also offers factchecking using Wikipedia APIs and allows users to log their conversation history and provide feedback on suggestions to improve personalization.

Built using FastAPI and Streamlit, the application ensures a responsive, interactive, and modular architecture. All components are unit-tested using pytest, with clearly defined backend services, routes, and a dynamic frontend.

Scenarios:

Scenario 1: Generating Smart Starters

A user enters an event description such as "AI for Sustainable Cities" and specifies interests like "climate change" and "urban planning." The assistant extracts themes (e.g., AI, sustainability) and generates 2–3 conversation starters aligned with the user's goals.

Scenario 2: Quick Fact Verification

A user preparing to attend a conference searches for a quick fact check on "blockchain in healthcare." The app provides a summarized, reliable reference using Wikipedia API.

Scenario 3: Reviewing Past Strategies

A user accesses the "History" feature to review previously generated conversations, and which ones were marked useful (thumbs up). This encourages continuous improvement in networking skills.

Technical Architecture:

Description: The Personalized Networking Assistant follows a modular AI-based architecture to provide intelligent networking recommendations. The frontend developed with Streamlit allows users to register, create profiles, and interact with AI-generated suggestions. The FastAPI backend validates requests and manages user data.

The AI engine uses DistilBERT to understand event themes, GPT-2 to generate conversation starters, and the Wikipedia API to verify facts before presenting recommendations. User profiles, conversation history, and feedback are stored in SQLite, enabling continuous personalization and improved recommendation quality. The application is designed for scalable deployment on IBM Cloud or Render.

Pre-requisites

- Python Programming
- FastAPI Framework
- Streamlit Framework
- Transformers Library
- PyTorch
- Hugging Face Models
- SQLite Database
- HTML & CSS
- Git & GitHub
- IBM Cloud / Render Deployment
- Visual Studio Code
- Virtual Environment (venv)

Project Workflow:

Milestone 1: AI Model Selection and System Architecture

Activity 1.1: Set Up the Development Environment

- Install Python, VS Code, and required libraries.
- Configure FastAPI and Streamlit.
- Install Transformers, PyTorch, Pandas, and SQLite.

Activity 1.2: Collect and Prepare Data

- Collect sample user profiles and networking event data.
- Clean and preprocess the data.
- Organize user interests and event information.

Activity 1.3: Select AI Models

- Study DistilBERT for event theme analysis.
- Study GPT-2 for conversation generation.
- Select Wikipedia API for fact verification.

Activity 1.4: Design the System Architecture

- Design frontend, backend, AI modules, and database.
- Define data flow between user, AI engine, and database.

Milestone 2: Core Functionalities Development

Activity 2.1: Develop User Management

- Implement user registration and login.
- Create profile management module.
- Store user information securely.

Activity 2.2: Develop AI Modules

- Integrate DistilBERT for event theme analysis.
- Integrate GPT-2 for personalized conversation starters.
- Connect Wikipedia API for fact verification.

Activity 2.3: Store User Data

- Save conversation history.

- Store user feedback.
- Maintain networking recommendations.

Milestone 3: Backend Development

Activity 3.1: Develop FastAPI Backend

- Create API routes.
- Validate user inputs.
- Process networking requests.

Activity 3.2: Integrate AI Services

- Connect AI models with backend.
- Generate conversation suggestions.
- Return personalized networking recommendations.

Activity 3.3: Database Integration

- Store user profiles.
- Save networking history.
- Retrieve previous conversations.

Milestone 4: Frontend Development

Activity 4.1: Design User Interface

- Create Home Page.
- Design Login and Registration pages.
- Build Profile Setup page.
- Create Event Details page.

Activity 4.2: Develop Interactive Features

- Display AI-generated conversation starters.
- Show verified information.
- Display networking recommendations.
- Provide conversation history.

Milestone 5: Deployment and Testing

Activity 5.1: Prepare the Application

- Install project dependencies.

- Configure environment variables.
- Test backend APIs.

Activity 5.2: Local Testing

- Run the application locally.
- Verify AI-generated responses.
- Test conversation history and recommendations.

Activity 5.3: Cloud Deployment

- Push source code to GitHub.
- Deploy on IBM Cloud or Render.
- Verify successful deployment.

Milestone 6: Final Testing and Documentation**Activity 6.1: System Testing**

- Test all application features.
- Verify AI recommendations.
- Validate fact verification results.

Activity 6.2: Project Completion

- Finalize documentation.
- Demonstrate the application.
- Submit the Personalized Networking Assistant project.

Milestone 1: AI Model Selection and Solution Architecture

In this milestone, we focus on selecting the appropriate Artificial Intelligence and Natural Language Processing models for the **Personalized Networking Assistant**. This involves setting up the development environment, collecting user profile and event data, preprocessing the data, selecting suitable AI models, and designing the overall system architecture. The architecture defines the interaction between the Streamlit user interface, FastAPI backend, AI models, database, and users.

Activity 1.1: Set Up the Development Environment

Before developing the Personalized Networking Assistant, install the required software and Python libraries.

Step 1 - Install Python and Anaconda

Download and install **Python 3.10 or above** and **Visual Studio Code**. These tools provide the environment required for AI model development, backend programming, and web application development.

```
PS C:\Users\DELL\Desktop\PersonalizedNetworkingAssistant\PersonalizedNetworkingAssistant> py --list
*
Active venv
-V:3.13      Python 3.13 (64-bit)
-V:3.12      Python 3.12 (64-bit)
-V:3.10      Python 3.10 (64-bit)
PS C:\Users\DELL\Desktop\PersonalizedNetworkingAssistant\PersonalizedNetworkingAssistant> python --version
Python 3.13.4
```

Step 2 - Install Required Libraries

```
pip install streamlit fastapi transformers torch pandas numpy sqlite3 wikipedia-api uvicorn
```

These libraries are used for user interface development, backend processing, Natural Language Processing, AI model integration, database management, and fact verification.

```
(venv) PS C:\Users\DELL\Desktop\PersonalizedNetworkingAssistant\PersonalizedNetworkingAssistant> pip list
```

Package	Version
altair	6.2.2
annotated-doc	0.0.4
annotated-types	0.7.0
anyio	4.14.1
attrs	26.1.0
beautifulsoup4	4.15.0
blinker	1.9.0
cachetools	7.1.4
certifi	2026.6.17
charset-normalizer	3.4.7
click	8.4.2
colorama	0.4.6
fastapi	0.139.0
filelock	3.29.5
fsspec	2026.6.0
gitdb	4.0.12
GitPython	3.1.50
h11	0.16.0
hf-xet	1.5.1
httpcore	1.0.9
httptools	0.8.0
httpx	0.28.1
huggingface_hub	1.22.0
idna	3.18
iniconfig	2.3.0
itsdangerous	2.2.0
Jinja2	3.1.6
joblib	1.5.3
jsonschema	4.26.0
jsonschema-specifications	2025.9.1
markdown-it-py	4.2.0
MarkupSafe	3.0.3
mdurl	0.1.2

```
(venv) PS C:\Users\DELL\Desktop\PersonalizedNetworkingAssistant\PersonalizedNetworkingAssistant> pip list
```

python-multipart	0.0.32
PyYAML	6.0.3
referencing	0.37.0
regex	2026.6.28
requests	2.34.2
rich	15.0.0
rpds-py	2026.6.3
safetensors	0.8.0
scikit-learn	1.9.0
scipy	1.18.0
sentence-transformers	5.6.0
setuptools	81.0.0
shellingham	1.5.4
six	1.17.0
smmap	5.0.3
soupsieve	2.8.4
starlette	1.3.1
streamlit	1.58.0
sympy	1.14.0
tenacity	9.1.4
threadpoolctl	3.6.0
tokenizers	0.22.2
toml	0.10.2
torch	2.12.1
tqdm	4.68.3
transformers	5.13.0
typer	0.26.8
typing_extensions	4.16.0
typing-inspection	0.4.2
tzdata	2026.2
urllib3	2.7.0
uvicorn	0.50.0
watchdog	6.0.0
websockets	16.0
wikipedia	1.4.0

Step 3 - Prepare User and Event Data

Collect sample user profile information and networking event details. The dataset includes user interests, professional skills, career goals, event topics, and networking preferences required for AI-based recommendations.

Step 4 - Select AI Models

Study and compare suitable AI and NLP models for the project.

- DistilBERT for Event Theme Analysis
- GPT-2 for Conversation Starter Generation
- Wikipedia API for Fact Verification

Based on performance and efficiency, **DistilBERT** and **GPT-2** were selected because they provide accurate topic understanding and natural conversation generation.

Step 5 - Configure AI Models

Download and configure the selected pre-trained AI models using the Hugging Face Transformers library. These models are integrated into the application to generate personalized conversation suggestions.

```
import joblib
joblib.dump(model, "floods.save")import joblib
```

Step 6 - Design the Solution Architecture

Design the overall system architecture illustrating the interaction between the User Interface, FastAPI Backend, AI Models, Database, and Personalized Networking Recommendations. This architecture forms the foundation for the application's implementation and deployment.

Activity 1.2: Research and Select the Appropriate AI Models

The following activities were carried out to identify the most suitable AI models for the Personalized Networking Assistant.

- Understand the project objective of improving professional networking using Artificial Intelligence.
- Study Natural Language Processing techniques for understanding event themes and user interests.
- Explore transformer-based models including **DistilBERT** and **GPT-2**.
- Evaluate the models based on response quality, speed, and relevance.
- Select **DistilBERT** for event theme analysis and **GPT-2** for personalized conversation generation.
- Integrate the **Wikipedia API** to verify factual information before presenting recommendations.

Activity 1.3: Define the Architecture of the Application

- Draft an architecture diagram showing the interaction between the Streamlit frontend, FastAPI backend, AI models, SQLite database, and external APIs.
- Design the user interface for registration, profile creation, event details, AI conversation suggestions, and networking history.
- Define how the FastAPI backend processes user requests, validates profile information, and communicates with AI models.
- Explain how DistilBERT analyzes event topics, GPT-2 generates conversation starters, and Wikipedia API verifies information before displaying results.
- Store user profiles, conversation history, and feedback in the SQLite database for future recommendations.

Activity 1.4: Set Up the Development Environment

- **Install Python and Pip:** Ensure **Python 3.10 or above** is installed along with **pip** for managing project dependencies.
- **Create a Virtual Environment:** Create and activate a virtual environment to isolate project dependencies.

```
C:\Users\DELL\Desktop\PersonalizedNetworkingAssistant> python -m venv venv
```

```
C:\Users\DELL\Desktop\PersonalizedNetworkingAssistant> venv\Scripts\activate
```

```
(venv) C:\Users\DELL\Desktop\PersonalizedNetworkingAssistant> pip install -r requirements.txt
```

- **Install Required Python Libraries:** Install all the required libraries for FastAPI, Streamlit, Artificial Intelligence, Natural Language Processing, Database Management, and Fact Verification.

```
(venv) C:\Users\DELL\Desktop\PersonalizedNetworkingAssistant> pip install fastapi streamlit transformers torch sentence-transformers pandas numpy wikipedia-api uvicorn python-dotenv pydantic
```

- **Verify the Installation:**
- Confirm that all required libraries are installed successfully before running the application.

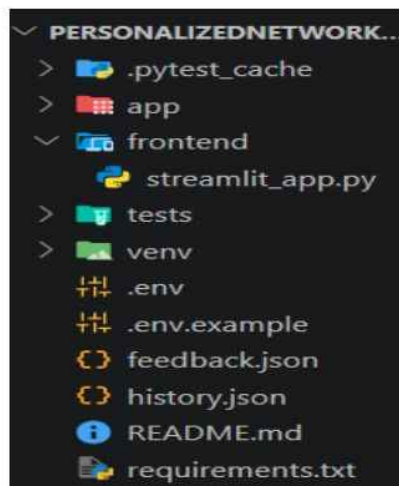
```
(venv) C:\Users\DELL\Desktop\PersonalizedNetworkingAssistant> pip list
```

- **Run the Flask Application:**
- Start the FastAPI development server to verify that the backend is working correctly.

```
Python -m uvicorn app.main:app --reload
```

- **Run the Streamlit Frontend:**
- Open another terminal and start the Streamlit application.
 - **python -m streamlit run frontend\streamlit_app.py**
- **Set Up the Project Structure:**

Organize the project directory with separate folders for the FastAPI backend, Streamlit frontend, AI services, routers, schemas, tests, and supporting project files.



Milestone 2: Core Functionalities Development

Activity 2.1 Develop Core Networking Assistant Features

Implement the core functionalities of the Personalized Networking Assistant, including user profile management, AI-powered conversation generation, event theme analysis, and fact verification through the Streamlit web application.

- **User Profile Management**

Collect and manage user information such as name, profession, interests, skills, career goals, and networking preferences. Validate the input data before processing.

- **AI Conversation Generation**

Integrate the GPT-2 model to generate personalized conversation starters based on the user's profile and event details, helping users initiate meaningful professional conversations.

- **Event Theme Analysis**

Use the DistilBERT model to analyze event topics and recommend relevant discussion points that match the user's interests.

- **Fact Verification Module**

Integrate the Wikipedia API to verify information related to discussion topics and provide accurate, reliable content during networking conversations.

- **Recommendation Result Generation**

Display personalized networking recommendations, AI-generated conversation starters, verified information, and event-related suggestions through a user-friendly web interface.

Activity 2.2: Implement the FastAPI Backend

Define Routes in Flask:

- Configure the required routes in **app.py** for the Home page, Prediction Input page, and Prediction Result page.

Process User Input:

- Create an input form in the HTML template to allow users to enter rainfall and weather-related parameters.
- Use Flask's **request.form** to receive user input from the prediction form.
- Validate that all required input values are entered correctly before processing the prediction request.

Integrate the Machine Learning Model:

- Load the trained **XGBoost model (floods.save)** using **Joblib/Pickle** during application startup.
- Pass the processed user input to the trained model for prediction.
- Receive the prediction result from the model and determine whether there is a **Flood Chance** or **No Flood Chance**.
- Display the prediction result dynamically on the result page using Flask's **render_template()** function.

This implementation ensures smooth interaction between the **Streamlit user interface**, **FastAPI backend**, **DistilBERT**, **GPT-2**, **Wikipedia API**, and the **SQLite database**, providing users with intelligent networking recommendations, personalized conversation starters, and verified information in real time.

Milestone 3: App.py Development

Milestone 3 focuses on developing and refining the core logic of the **main.py** and **conversation.py** files, which serve as the backbone of the **Personalized Networking Assistant** application. During this milestone, the **FastAPI backend** and **Streamlit frontend** are configured, AI services are integrated, user inputs are validated, and personalized networking recommendations are generated and displayed through the web interface.

Activity 3.1: Writing the Main Application Logic in app.py

Define the Core Routes in app.py

Establish the required API routes for the Personalized Networking Assistant.

Home Route (/)

```
@app.get("/")
async def read_root():
    return {
        "message": "Welcome to the Personalized Networking Assistant API!",
        "status": "online"
    }
```

Generate Conversation Route

/generate-conversation — POST endpoint that accepts user profile and event information, analyzes event themes, generates AI-powered conversation starters, stores conversation history, and returns personalized networking recommendations.

```
@router.post("/generates-conversation")

Async def generate_conversation(data:
ConversationRequest):
```

Set Up Route Handlers for Each Feature:

For the /generate-conversation Route

- Read user profile information such as **Name, Profession, Skills, Interests, Career Goals, and Networking Goals**.
- Read event information including **Event Name** and **Event Description**.
- Validate all user inputs before processing.
- Extract event themes using the **DistilBERT** model.
- Generate personalized conversation starters using **GPT-2**.
- Save generated conversations to the conversation history.
- Return AI-generated networking recommendations to the Streamlit interface.

Event Theme Analysis

The application analyzes networking event descriptions and identifies the most relevant discussion themes using the DistilBERT model.

```
themes = extract_event_themes(  
    data.event_context.event_description  
)
```

Conversation Generation

Generate personalized networking conversation starters using the user's profile and extracted event themes.

```
starters = generate_topics(  
    data.user_profile,  
    data.event_context,  
    themes  
)
```

Conversation Generation

Store user profile information, detected themes, and generated conversation starters for future reference.

```
log_conversation(  
    data.user_profile.model_dump(),  
    themes,  
    starters  
)
```

Fact Verification

Verify networking topics using the Wikipedia API before presenting recommendations.

```
result = fact_check(data.query)
```

Define AI Service Loading

Load the required AI services during application startup.

```
from app.services.event_analyzer import extract_event_themes  
from app.services.topic_generator import generate_topics  
from app.services.fact_checker import fact_check
```


Integrate the AI Models

```
themes =  
    extract_event_t  
    hemes(data.event  
    t_context.event  
    _description)  
  
starters =  
    generate_topics  
    (data.user_prof  
    ile,data.event_  
    context,themes  
    )  
  
log_conversatio  
n(data.user_pro  
file.model_dump  
(),themes,start  
ers  
)
```

Integrate the AI Models

The application uses try-except blocks to handle invalid inputs, AI processing errors, and API failures, returning meaningful error messages to the user.

```
try:  
    # Process AI recommendations  
except Exception as e:  
    logger.error(f"Error: {e}")  
    raise HTTPException(status_code=500, detail=str(e))
```

Milestone 4: Frontend Development

Milestone 4 focuses on developing a simple, responsive, and user-friendly web interface for the Personalized Networking Assistant. The frontend is built using Streamlit, HTML, and CSS to collect user profile information, event details, and display AI-generated networking recommendations, conversation starters, and verified information in an easy-to-understand format.

Activity 4.1: Designing and Developing the User Interface

Set Up the Base HTML Structure:

- Develop the main Home Page to introduce the Personalized Networking Assistant with navigation to the Login and Dashboard pages.
- Create a User Profile Page containing a form for entering user details such as Name, Profession, Skills, Interests, Career Goals, and Event Information.
- Design separate pages for AI Conversation Suggestions, Fact Verification, Networking Recommendations, and Conversation History.

Design a Responsive Layout Using CSS:

- Create a CSS stylesheet to provide a clean, responsive, and user-friendly interface.
- Use responsive layouts so the application works properly on desktop, tablet, and mobile devices.
- Apply appropriate colors, buttons, icons, input fields, and spacing to improve usability.

Create Separate UI Components :

- **Home Page:** Displays project information and navigation.
- **Login & Registration Page:** Allows users to create an account and log in securely.
- **User Profile Page:** Collects user information and networking preferences.
- **Event Details Page:** Allows users to enter conference or event information.
- **AI Suggestions Page:** Displays personalized conversation starters and networking recommendations.
- **Fact Verification Page:** Displays verified information using the Wikipedia API.
- **Conversation History Page:** Shows previous conversations and saved recommendations.

Activity 4.2: Creating Dynamic Content Rendering

Handle Form Submission :

- Create forms using **Streamlit** to collect user profile details and event information.
- Submit user input to the **FastAPI backend** for processing.
- Validate all required fields before sending the request.

Display AI Recommendation Results:

- Receive personalized networking recommendations from the AI models.
- Display AI-generated conversation starters.
- Show verified information and event-related discussion topics.
- Present networking recommendations in a clear and user-friendly format.

Improve User Experience:

- Ensure smooth navigation between Home, Login, Profile, Dashboard, and Recommendation pages.
- Display meaningful error messages if invalid or incomplete information is entered.

- Maintain a simple, responsive, and accessible interface across all devices.
- Provide quick access to conversation history and user feedback.
- Ensure fast loading and real-time response for AI-generated suggestions.

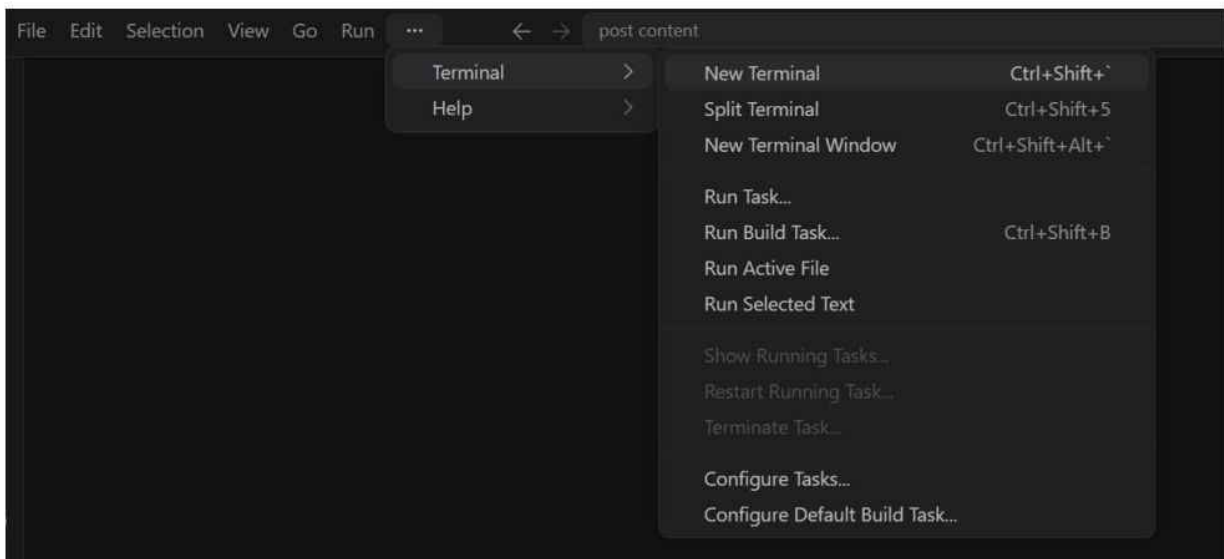
Milestone 5: Deployment

Milestone 5 focuses on deploying the Personalized Networking Assistant application locally and on the cloud to verify its functionality. This includes setting up the project environment, installing the required dependencies, running the FastAPI backend and Streamlit frontend, testing AI-powered networking recommendations, and deploying the application using Render or IBM Cloud for public access.

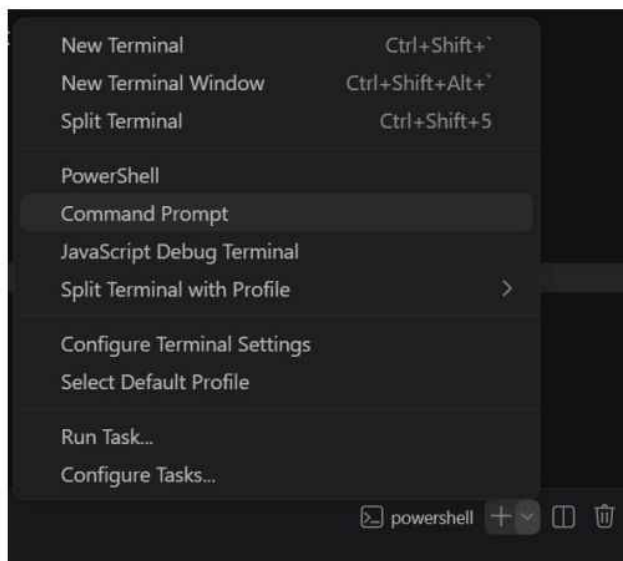
Activity 5.1: Preparing the Application for Local Deployment

Set Up a Virtual Environment:

- Open a New Terminal
- Open **Command Prompt** or **PowerShell**.
- Navigate to the project directory

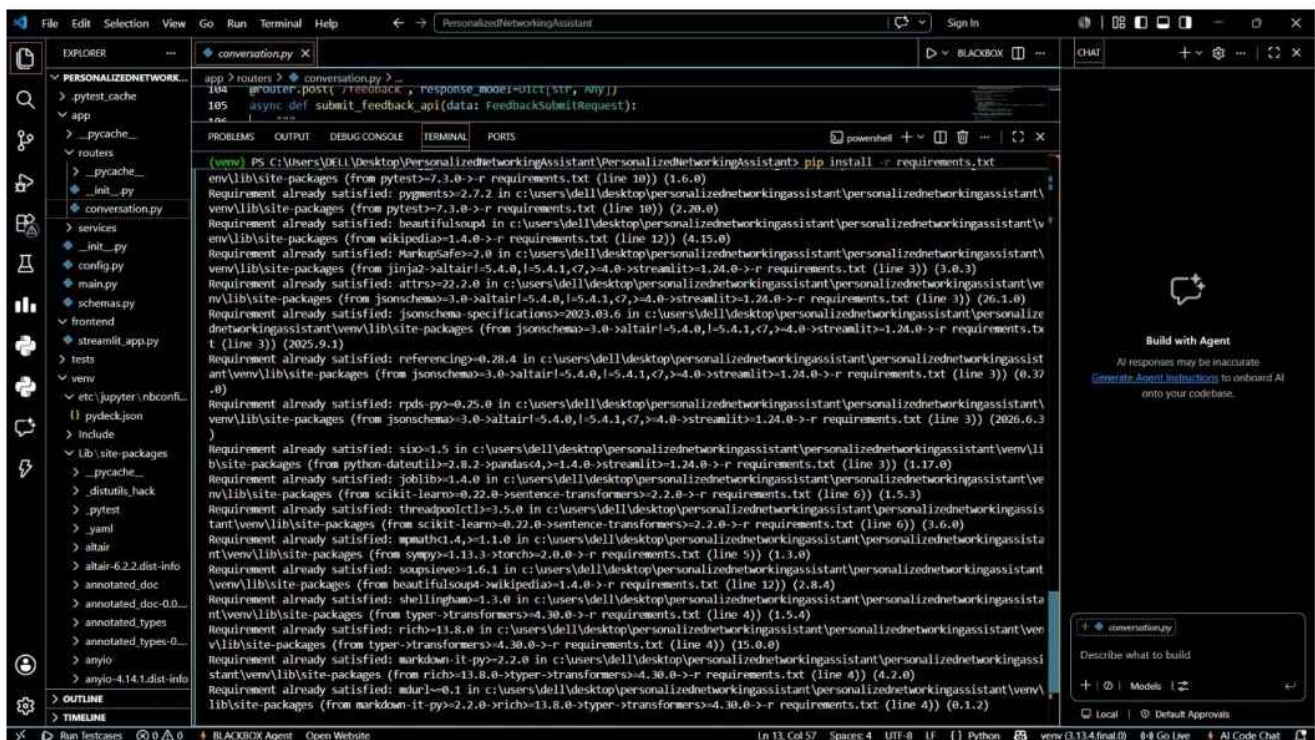


- Then open "Command Prompt"



- Navigate to the project directory.
- Install all the required dependencies using the following command:

```
pip install -r requirements.txt
pip install -r requirements.txt
```



This installs all the required Python libraries such as:

- FastAPI
- Streamlit
- Transformers
- PyTorch
- Pandas
- NumPy
- SQLite
- Wikipedia API
- Uvicorn
- Sentence Transformers

Activity 5.2: Local Testing and Verification

Start the Flask Development Server:

Start the FastAPI Backend

```
❏ Bash
uvicorn main:app --reload
```

Start the Streamlit Frontend

Open another terminal.

```
❏ Bash
streamlit run app.py
```

Open the Application

Open your browser and navigate to

```
http://localhost:8501
```

Verify the Application

Test the following features:

- User Profile Creation
- Event Details Submission
- Event Theme Analysis
- AI Conversation Starter Generation
- Fact Verification using Wikipedia API
- Networking Recommendations
- Conversation History
- User Feedback

Verify that AI-generated conversation starters and networking recommendations are displayed correctly.

Activity 5.3: Cloud Deployment Using Render

The Personalized Networking Assistant application was successfully deployed on the Render Cloud Platform to provide public access over the internet. Cloud deployment enables users to access the application without installing Python or project dependencies on their local systems.

Deployment Steps

- Push the project source code to the GitHub repository.
- Create a new **Web Service** in Render.
- Connect the GitHub repository.
- Set the **Root Directory** to:

Personalized-Networking-Assistant

- Configure the Build Command: **pip install -r requirements.txt**
- Configure the Start Command: **uvicorn main:app --host 0.0.0.0 --port 10000**
- For Streamlit: **streamlit run app.py --server.port=10000 --server.address=0.0.0.0**
- Deploy the application and verify successful deployment.

Deployment Verification

- Open the deployed application using the Render URL.
- Register or Login.
- Enter user profile information.
- Enter networking event details.
- Generate AI conversation starters.
- Verify networking recommendations.
- Check fact verification results.
- Verify conversation history and feedback modules.

Deployment URL: <https://rising-waters-hh79.onrender.com/>

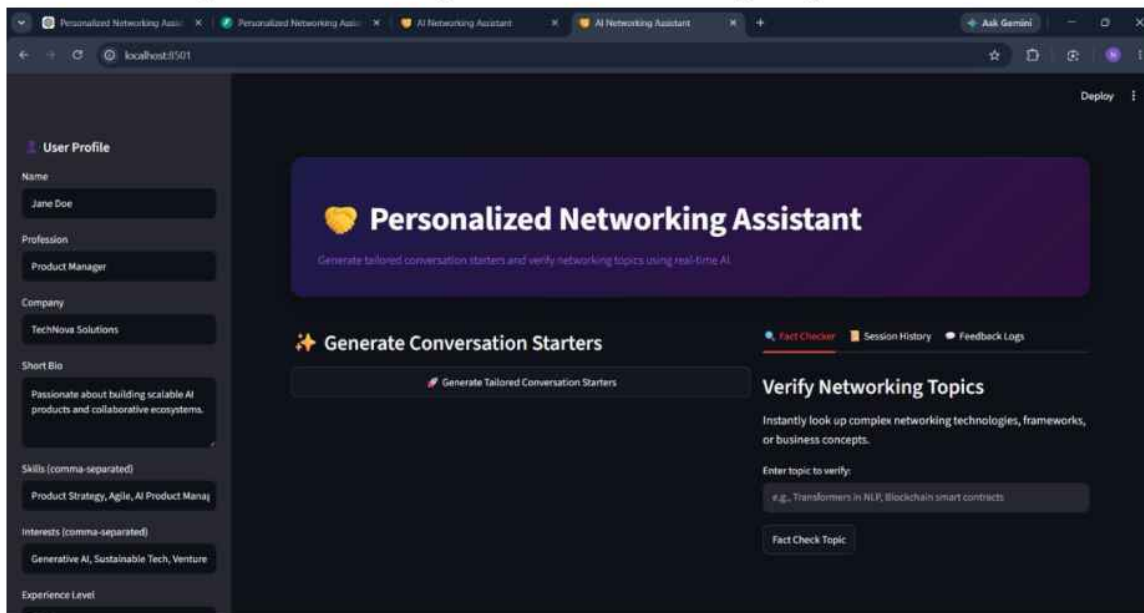
Outcome

The **Personalized Networking Assistant** was successfully deployed on the Render cloud platform and is accessible through any web browser. The deployed application generates personalized conversation starters, analyzes event themes using **DistilBERT**, verifies information using the **Wikipedia API**, recommends networking opportunities, and stores conversation history using SQLite.

Exploring the Website's Web Pages:

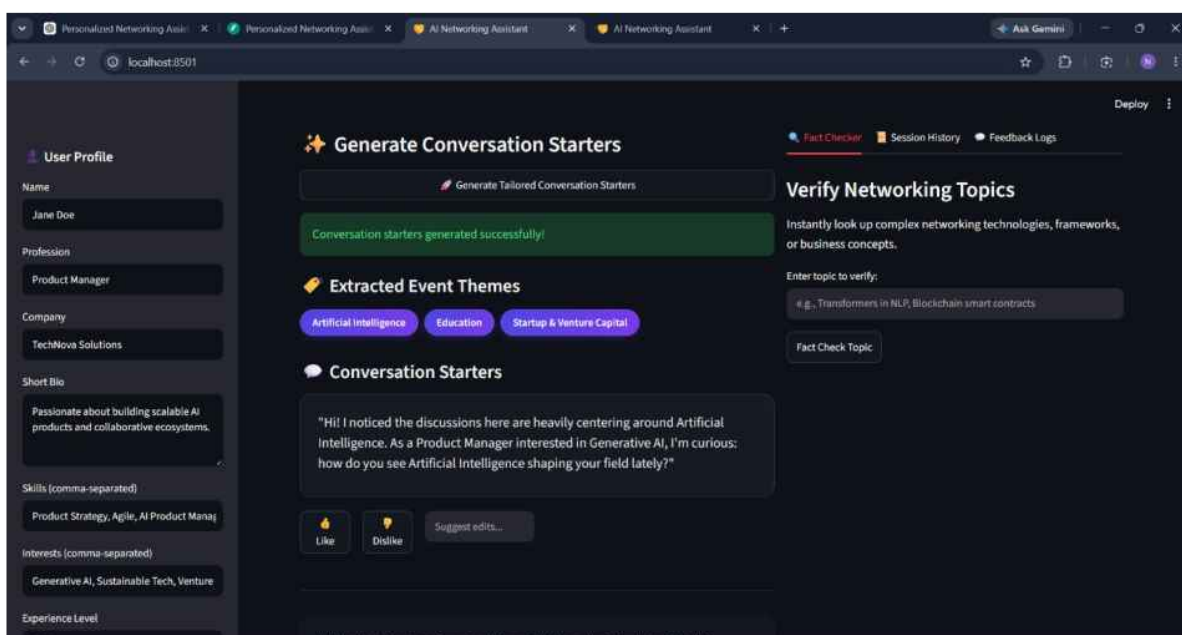
Home / Generator Page:

The Home Page serves as the landing page of the Personalized Networking Assistant. It introduces the application, explains its purpose, and provides navigation to the Login and Dashboard pages. Users can learn about the AI-powered networking features before accessing the system.



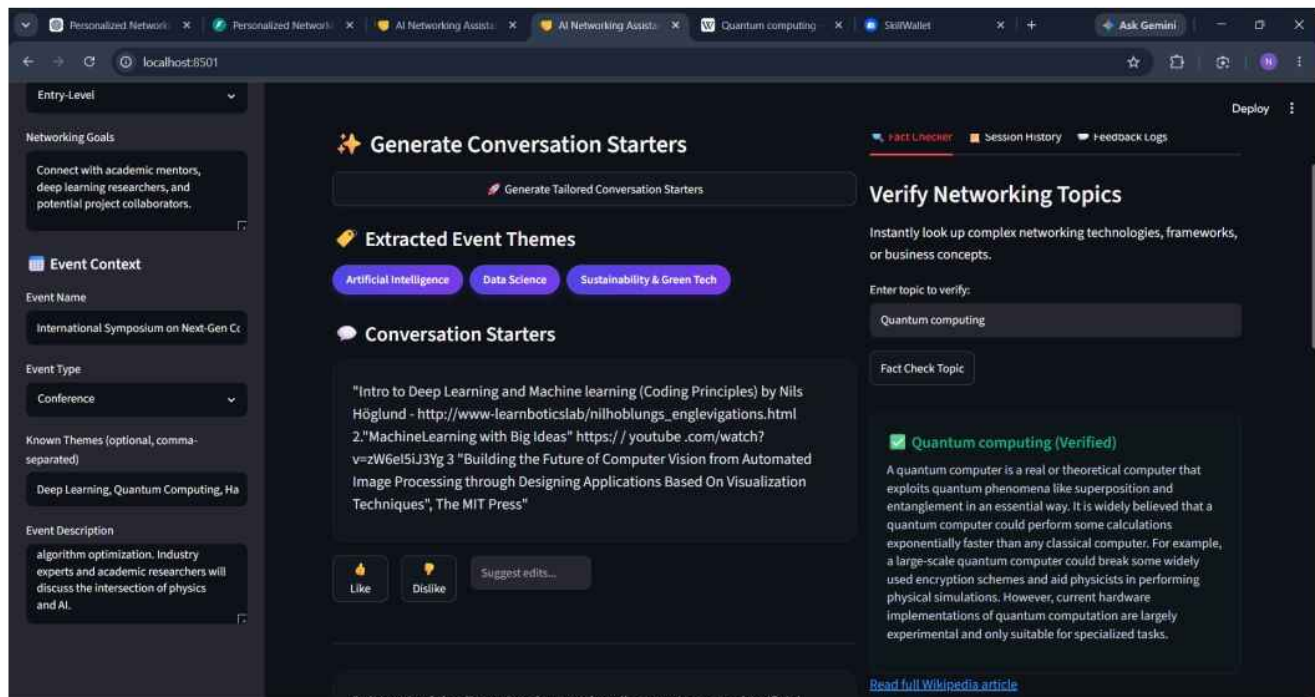
Input Section:

The User Profile Page allows users to enter professional information such as name, profession, skills, interests, career goals, and networking event details. This information is used by the AI models to generate personalized recommendations.



Results Section:

The AI Recommendation Page displays AI-generated conversation starters, recommended networking topics, event theme analysis, and verified information. DistilBERT analyzes event themes, GPT-2 generates conversation starters, and the Wikipedia API verifies factual information before displaying recommendations.



Conclusion:

The **Personalized Networking Assistant** is an AI-powered web application developed using **FastAPI**, **Streamlit**, **DistilBERT**, **GPT-2**, and the **Wikipedia API**. The system generates personalized conversation starters, analyzes networking event themes, verifies factual information, and recommends meaningful professional connections. The modular architecture, AI integration, and responsive user interface provide an efficient networking experience for professionals, students, and conference participants.

Future enhancements include **LinkedIn integration**, **real-time event APIs**, **cloud scalability**, **multilingual AI support**, **mobile application development**, and **advanced recommendation algorithms** to further improve user experience and networking effectiveness.